

PURDUE UNIVERSITY

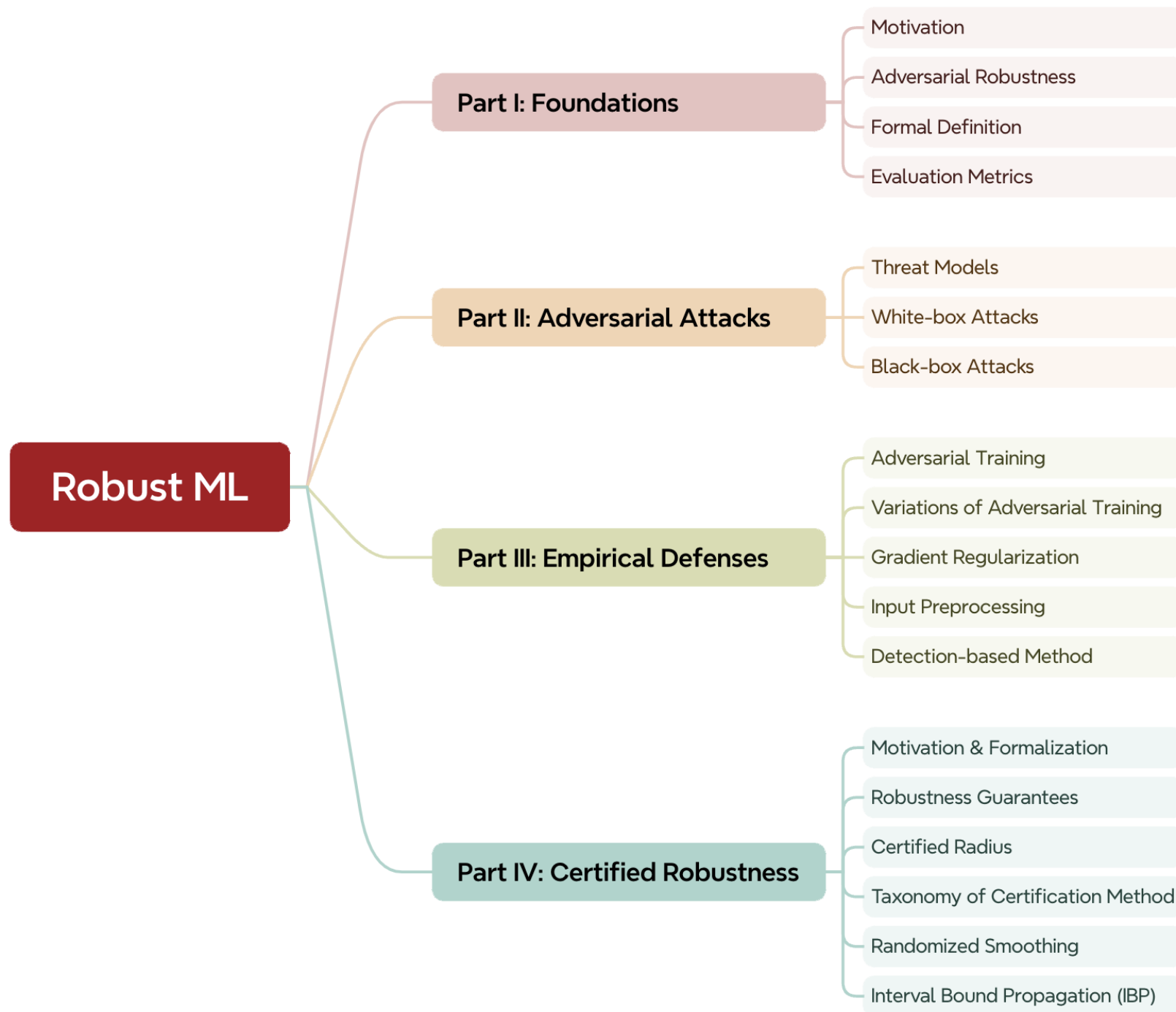
DEPARTMENT OF COMPUTER SCIENCE

Robustness in Machine Learning

From Adversarial Robustness to Certified Robustness

Elisa Bertino, Mengdie (Maggie) Huang

February 17 2026



Part I | Foundations



What is robustness in machine learning?

❖ Robustness

- Refers to the ability of a system or model to maintain stable and reliable performance across a broad spectrum of conditions, variations, or challenges, demonstrating resilience and adaptability in the face of uncertainties or unexpected changes.
- For instance, we consider a sensor robust if it is resilient to disturbances from the environment.

❖ Definition of Robustness in ML

ML Model robustness denotes the capacity of a model to sustain stable predictive performance in the face of variations and changes in the input data.

- The model f is considered robust if its output remains unchanged after the input x is subjected to constrained unintentional or intentional perturbations.
- Formal Definition: For a given input x and its true label y , the model is locally robust if

$$f(x + \delta) = y, \quad \forall \delta \in \Delta$$

- δ : is the perturbation and Δ is the set of allowed perturbations (e.g., $\|\delta\|_p \leq \epsilon$).

What is robustness in machine learning?

❖ Robustness

- Refers to the ability of a system or model to maintain stable and reliable performance across a broad spectrum of conditions, variations, or challenges, demonstrating resilience and adaptability in the face of uncertainties or unexpected changes.
- For instance, we consider a sensor robust if it is resilient to disturbances from the environment.

❖ Definition of Robustness in ML

Generalization: "How does the model perform on the **average case from the test set**?"

Robustness: "How does the model perform on the **worst case within a specific neighborhood**?"

- The model f is considered robust if its output remains unchanged after the input x is subjected to constrained unintentional or intentional perturbations.
- Formal Definition: For a given input x and its true label y , the model is locally robust if

$$f(x + \delta) = y, \quad \forall \delta \in \Delta$$

- δ : is the perturbation and Δ is the set of allowed perturbations (e.g., $\|\delta\|_p \leq \epsilon$).

Why study robustness?

❖ Safety-Critical Systems

- In safety-critical domains, a model's failure can threaten life. We need more than test accuracy.
 - Self-Driving: Misclassify a Stop sign as Speed Limit due to weather or an adversarial sticker.
 - Medical: Sensor noise or variations in imaging equipment lead to incorrect diagnoses.
 - Attackers craft adversarial perturbations to bypass facial recognition or malware detectors.

❖ Bridging the Generalization Gap

- Robustness is the bridge between toy models and trustworthy systems. Standard ML assumes that training and test data come from the same distribution, which always fails in the real world.
 - Distribution Shift: A model trained on daytime photos may fail in the rain or at night.
 - Spurious Correlations: Models often "cheat" by learning background patterns rather than the actual object. Robustness forces models to learn causal, invariant features

Focus	Question
Natural Noise	"Will this self-driving car crash in the fog?"
Adversaries	"Can a hacker bypass my facial recognition?"
Distribution Shift	"Will the model work at Hospital B if trained at Hospital A?"

Taxonomy of Robustness in Machine Learning

❖ Adversarial Robustness

- against **malicious input perturbations** intentionally introduced by attackers to fool the model.

❖ Natural (Corruption) Robustness

- against **non-malicious stochastic variations** in input features that challenge the learned pattern.

❖ Distributional Robustness

- against the **shift in the data distribution** between training and inference.

❖ Behavioral (Semantic) Robustness

- A (LLM) model's logical reasoning is stable across different but semantically equivalent inputs.

Type	Goal	Perturbation Source	Mathematical Metric
Adversarial	Defend against attacks	Worst-case optimization	Adversarial Acc / Certified Radius
Natural	Maintain reliability	Environmental noise	Corruption Acc
Distributional	Handle new domains	Dataset shift	Divergence (KL)
Behavioral	Ensure consistency	Semantic variation	Hallucination Rate

Taxonomy of Robustness in Machine Learning

❖ Adversarial Robustness

- against **malicious input perturbations** intentionally introduced by attackers to fool the model.

❖ Natural (Corruption) Robustness

- against **non-malicious stochastic variations** in input features that challenge the learned pattern.

❖ Distributional Robustness

- against the **shift in the data distribution** between training and inference.

❖ Behavioral (Semantic) Robustness

- A (LLM) model's logical reasoning is stable across different but semantically equivalent inputs.

Type	Goal	Perturbation Source	Mathematical Metric
Adversarial	Defend against attacks	Worst-case optimization	Adversarial Acc / Certified Radius
Natural	Maintain reliability	Environmental noise	Corruption Acc
Distributional	Handle new domains	Dataset shift	Divergence (KL)
Behavioral	Ensure consistency	Semantic variation	Hallucination Rate

Adversarial Robustness

❖ Definition

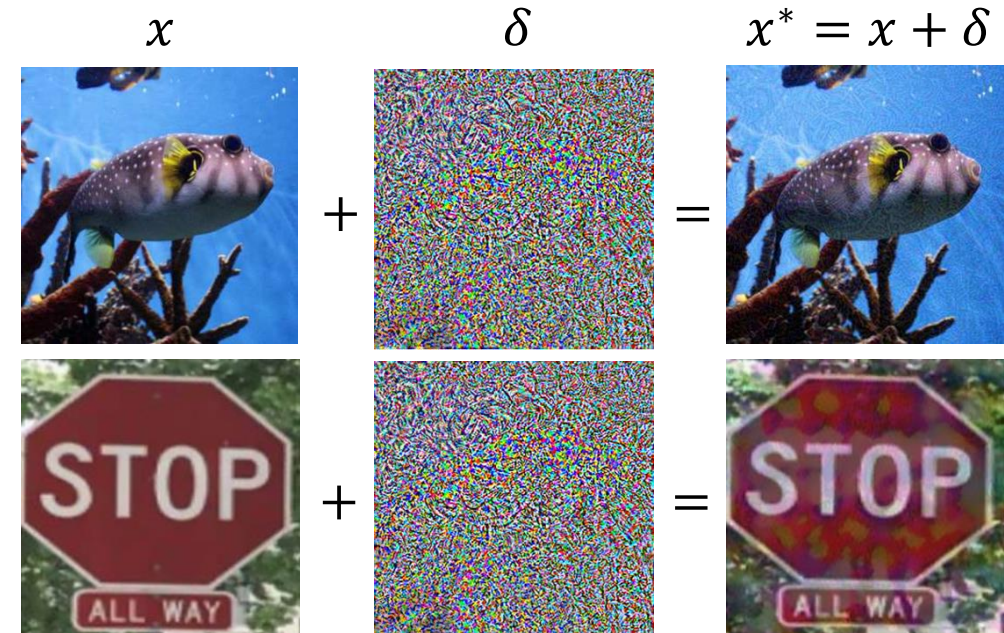
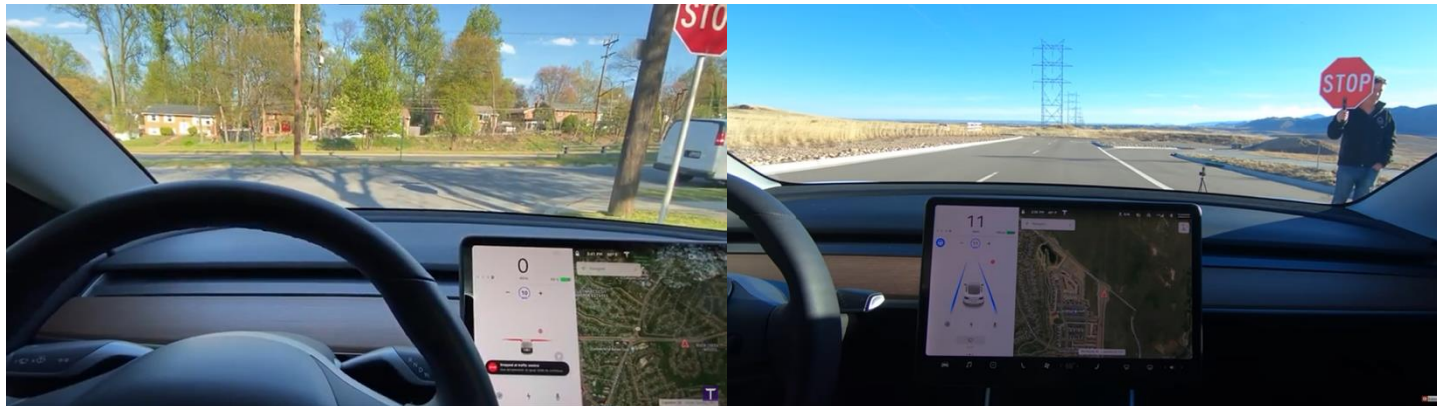
- The model's ability to maintain its output (prediction) for a given input when that input is subjected to a "worst-case" perturbation within a specific constraint set.

❖ Formalization of Adversarial Perturbation

- For a clean input x and its true label y , a classifier $f: \mathcal{X} \rightarrow \mathcal{Y}$ is considered locally robust at x if

$$f(x + \delta) = y, \forall \delta \in \Delta$$

- Perturbation δ : The change added to x .
- Constraint set Δ : The set of all allowed perturbations, e.g., $\Delta = \|\delta\|_p \leq \epsilon$.



Adversarial Robustness

❖ Evaluation Metrics

- Empirical metrics
 - Provides the **upper bound** of the true adversarial robustness of the evaluated model.
- Provable metrics
 - Provides the **lower bound** on true adversarial robustness of the evaluated model.

Evaluation Metric	Formula	Description
Clean Accuracy	$Acc_{cle} = \mathbb{E}_{(x,y) \sim \mathcal{D}}[\mathbb{I}(f(x) = y)]$	The accuracy on the original, clean test set.
Adversarial Accuracy	$Acc_{adv} = \mathbb{E}_{(x,y) \sim \mathcal{D}}[\mathbb{I}(\forall \delta \in \Delta, f(x + \delta) = y)]$	The accuracy when the input x is subject to a worst-case perturbation δ within a set Δ (e.g., an L_p ball).
Adversarial Gap	$Acc_{cle} - Acc_{adv}$	The drop in performance caused by adversarial attacks.
Average Distortion	$\mathbb{E}_{(x) \sim \mathcal{D}}[\min_{\delta: f(x+\delta) \neq y} \ \delta\ _p]$	The average minimum distance required for an attacker to flip the model's prediction.
Certified Radius	$R(x) = \sup \left\{ \epsilon: \forall \ \delta\ _p \leq \epsilon, f(x + \delta) = y \right\}$	The largest perturbation budget around x within which the prediction is mathematically proven not to change.
Certified Accuracy	$Acc_{cer} = \mathbb{E}_{(x,y) \sim \mathcal{D}}[\mathbb{I}(f(x) = y \wedge R(x) \geq \epsilon_{threshold})]$	The fraction of the test set that is provably robust against any attack with a magnitude up to $\epsilon_{threshold}$.

Part II | Adversarial Attacks



Taxonomy of Threat Models

❖ Attacker's Knowledge

White-Box



Full Knowledge

- Model architecture
- Training dataset
- Model parameters θ
- Model gradients $\nabla_x \mathcal{L}$
- Defenses used

Grey-Box



Partial Knowledge

- Model architecture or Training dataset

Black-Box



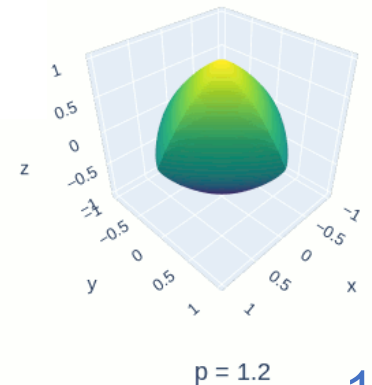
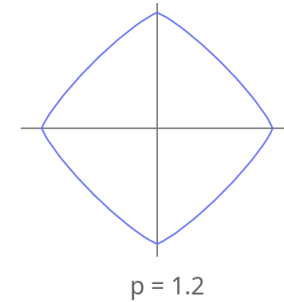
Zero Knowledge

- Input and output access only
 - Score-based (Logits)
 - Decision-based (Hard Label)
- Attacks rely on Transferability or Query-based estimation

❖ Attacker's Capabilities

- Norm-Bounded Constraints
 - The perturbation δ is restricted by an L_p norm

- L_∞ Norm: Limits the maximum change to any single feature. $\|x\|_\infty = \max_i |x_i|$
- L_2 Norm: Limits the total distance of the change. $\|x\|_2 = (\sum_{i=1}^n x_i^2)^{1/2}$
- L_0 -Norm: Limits the number of features modified. $\|x\|_0 = |\{i: x_i \neq 0\}|$



Taxonomy of Threat Models

❖ Attacker's Capabilities

- Semantic Perturbations
 - Modifying properties like **rotation, translation, color shifts**, or paraphrasing and synonym substitution in LLMs.
- Physical-World Constraints
 - Attacks that must survive real-world sensors, such as adversarial patches on clothing or stickers on traffic signs.

❖ Attacker's Goals

- Untargeted Attack $f(x + \delta) \neq y$
 - The goal is to make the model fail. The specific incorrect class doesn't matter.
- Targeted Attack $f(x + \delta) = y_{target} \neq y$
 - The goal is to force the model to predict a specific, incorrect class y_{target} .

❖ Attack Timing

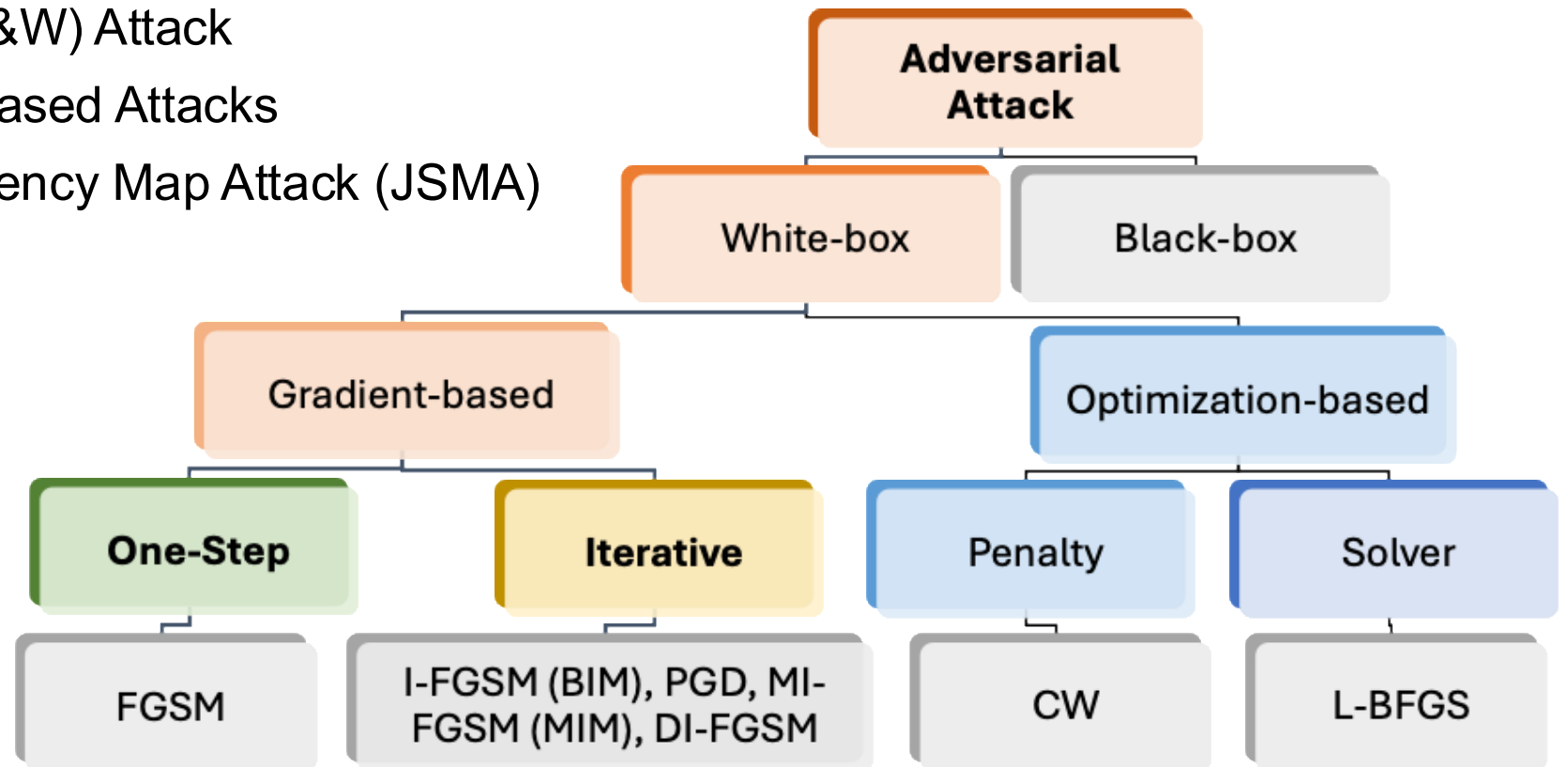
- Inference time: Evasion Attack
- Training time: Poisoning Attack

Adversarial robustness state: "The model achieves specific adversarial accuracy (%) against a white-box L_∞ adversary with $\epsilon = 8/255$."

White-Box Attack

❖ White-box Attack

- Gradient-Based Attacks
 - Fast Gradient Sign Method (FGSM)
 - Projected Gradient Descent (PGD)
- Optimization-Based Attacks
 - Carlini & Wagner (C&W) Attack
- Saliency and Decision-Based Attacks
 - Jacobian-based Saliency Map Attack (JSMA)
 - DeepFool

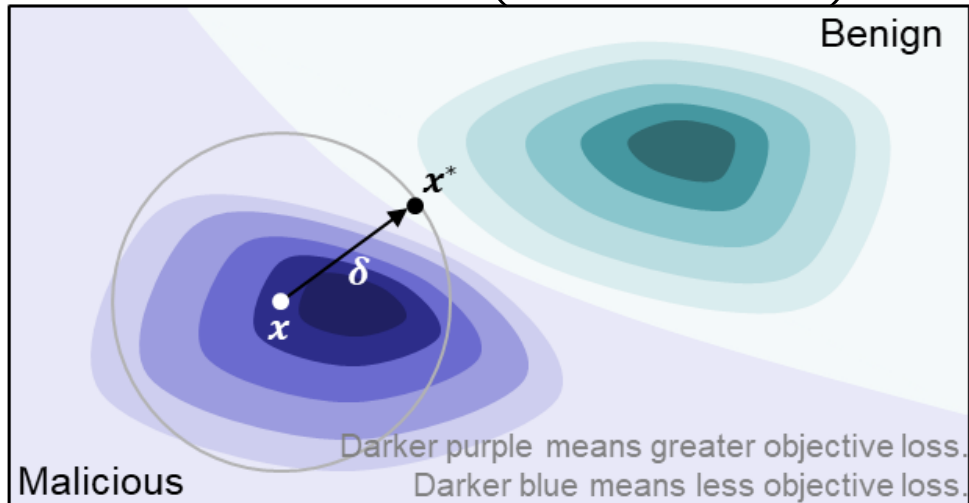


White-Box Attack

❖ Gradient-Based Attacks

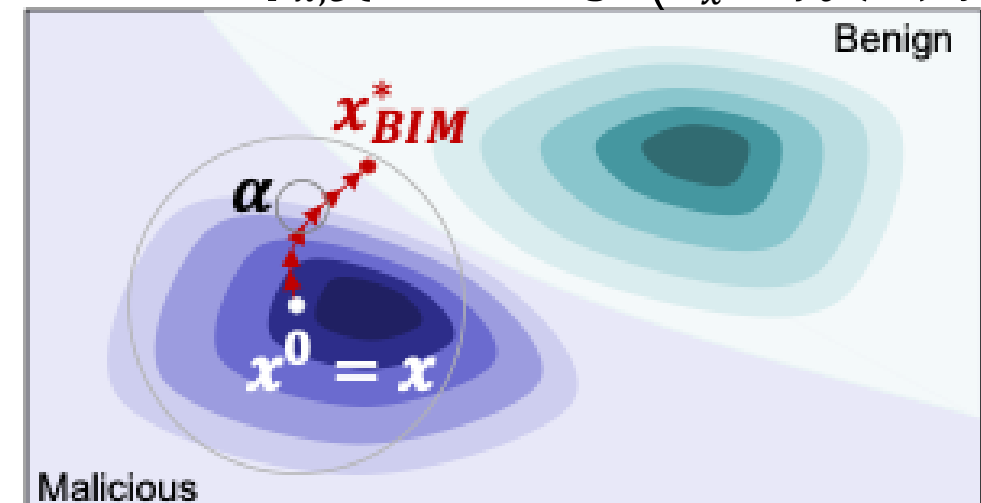
- Fast Gradient Sign Method (FGSM)
 - A one-step attack that moves the input in the direction of the gradient of the loss function.
 - "Hello World" of adversarial attacks.

$$x^* = x + \epsilon \cdot \text{sgn}(\nabla_{x^t} L(f_\theta(x), y))$$



- BIM (Basic Iterative Method)
 - An iterative version of FGSM that takes multiple small steps and projects the result back onto the L_p ball after each step.
 - Starts at the clean input $x^0 = x$

$$\text{sgn}(x) = \begin{cases} -1, & x < 0 \\ +1, & x > 0 \end{cases} \quad \text{Clip}_{x,\epsilon} \text{ used to project } x' \text{ onto } l_p\text{-ball}$$
$$x^{t+1} = \text{Clip}_{x,\epsilon} \{x^t + \alpha \cdot \text{sgn}(\nabla_{x^t} L(f_\theta(x^t), y))\}$$



White-Box Attack

❖ Gradient-Based Attacks

- Fast Gradient Sign Method (FGSM)
 - A one-step attack that moves the input in the direction of the gradient of the loss function.
 - "Hello World" of adversarial attacks.

- BIM (Basic Iterative Method)
 - An iterative version of FGSM that takes multiple small steps and projects the result back onto the L_p ball after each step.
 - Starts at the clean input $x^0 = x$



(a) Clean

(b) FGSM $l_\infty=32$

(c) BIM $l_\infty=32$

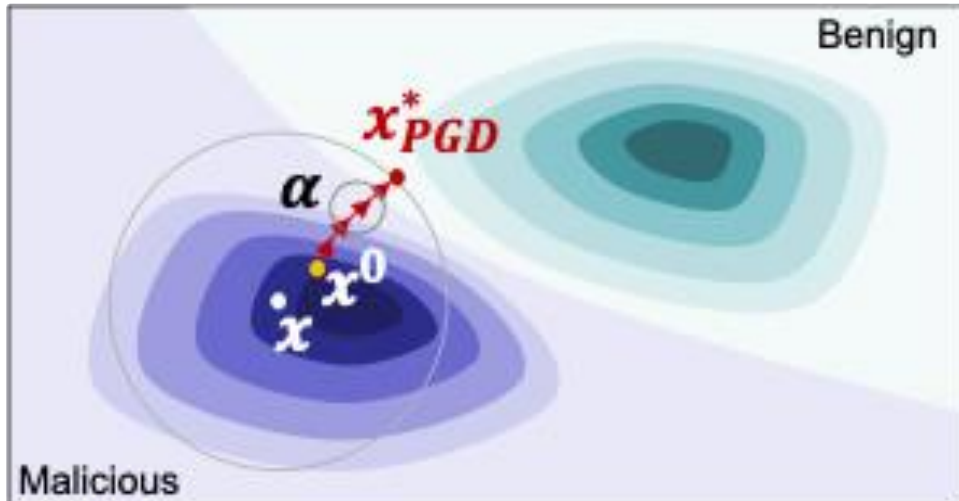
White-Box Attack

❖ Gradient-Based Attacks

- Projected Gradient Descent (PGD)
 - An iterative version of FGSM.
 - Starts at a random point within the L_p ball surrounding x . $x^0 = x + U(-\epsilon, \epsilon)$

$\text{sgn}(x) = \begin{cases} -1, & x < 0 \\ +1, & x > 0 \end{cases}$ $\Pi_{x+\Delta}(x')$ used to project x' onto l_p -ball

$$x^{t+1} = \Pi_{x+\Delta}(x^t + \alpha \cdot \text{sgn}(\nabla_{x^t} L(f_\theta(x^t), y)))$$

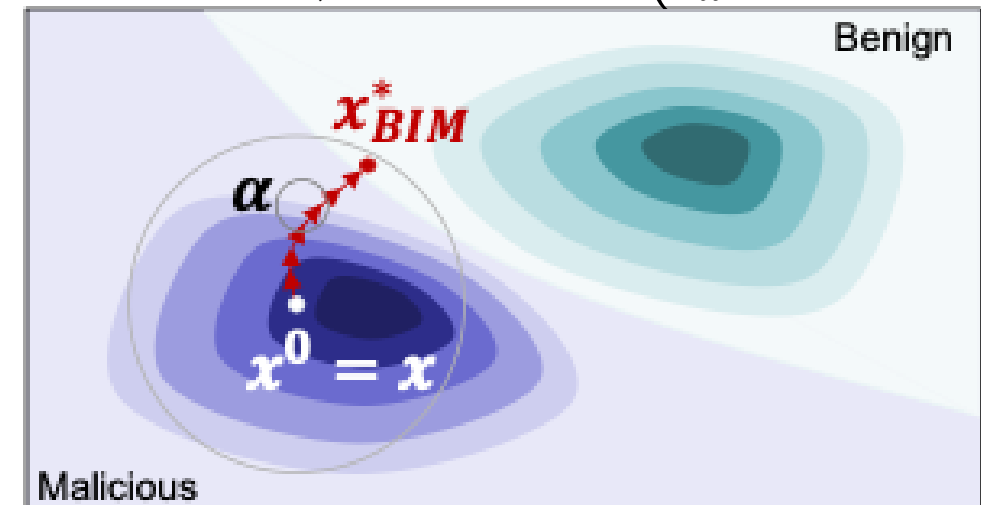


▪ BIM (Basic Iterative Method)

- An iterative version of FGSM that takes multiple small steps and projects the result back onto the L_p ball after each step.
- Starts at the clean input $x^0 = x$

$\text{sgn}(x) = \begin{cases} -1, & x < 0 \\ +1, & x > 0 \end{cases}$ $\text{Clip}_{x,\epsilon}$ used to project x' onto l_p -ball

$$x^{t+1} = \text{Clip}_{x,\epsilon}\{x^t + \alpha \cdot \text{sgn}(\nabla_{x^t} L(f_\theta(x^t), y))\}$$



White-Box Attack

❖ Saliency and Decision-Based Attacks

- Jacobian-based Saliency Map Attack (JSMA)
 - Targeted Attack, L_0 Attack
 - It relies on the Jacobian matrix $J_{i,j}$ of the model output $f_\theta(x)$ w.r.t the input x . the Jacobian matrix $J_{i,j}$ is the matrix of all first-order partial derivatives

$$J_{ij} = \nabla_x f_\theta(x) = \left[\frac{\partial f_{\theta_i}(x)}{\partial x_j} \right]_{i,j}$$

- JSMA identifies which specific pixels (features) have the highest impact on the output and modifies only those.
- DeepFool
 - Unlike attacks like PGD which stay within a fixed radius, DeepFool "pushes" the input until it just crosses the decision boundary. Thus, no fixed ϵ , it finds the minimum ϵ needed.

□ Papernot, Nicolas, Patrick McDaniel, Somesh Jha, Matt Fredrikson, Z. Berkay Celik, and Ananthram Swami. "The limitations of deep learning in adversarial settings." In 2016 IEEE European symposium on security and privacy (EuroS&P), pp. 372-387. IEEE, 2016.

□ Moosavi-Dezfooli, Seyed-Mohsen, Alhussein Fawzi, and Pascal Frossard. "Deepfool: a simple and accurate method to fool deep neural networks." In Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 2574-2582. 2016.

White-Box Attack

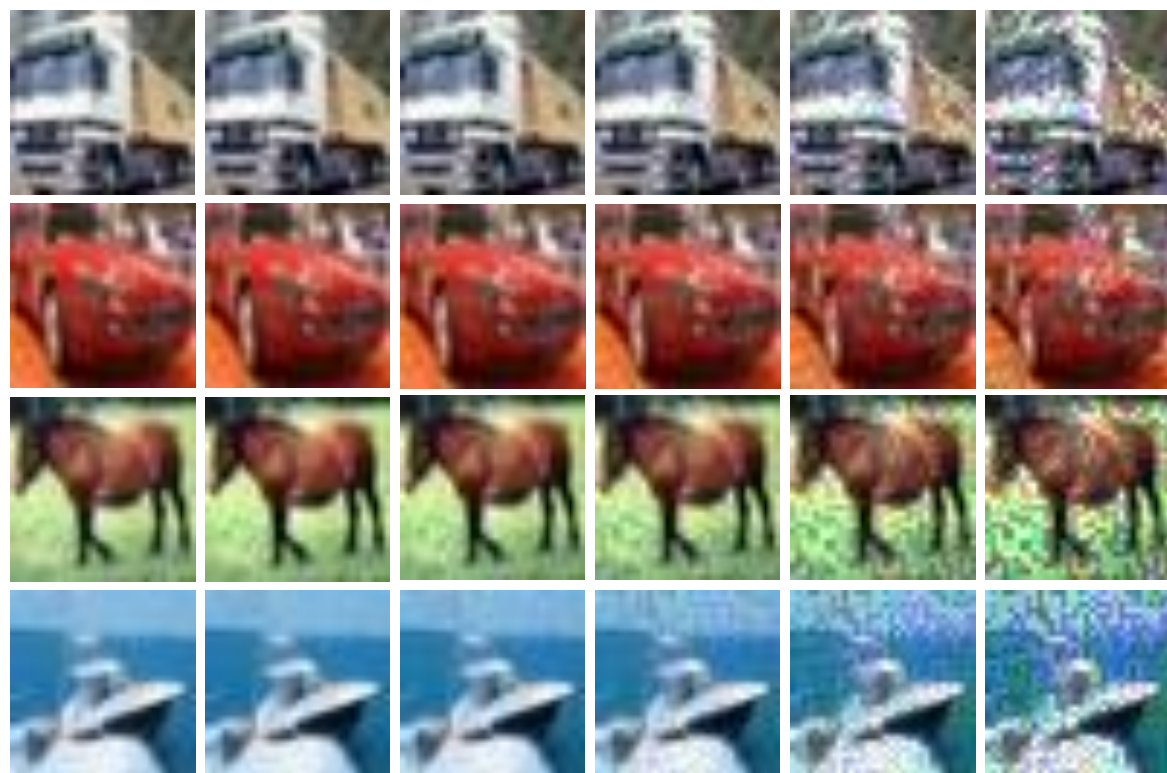


Benign

FGSM

BIM

CW



Clean

$\epsilon=0.02$

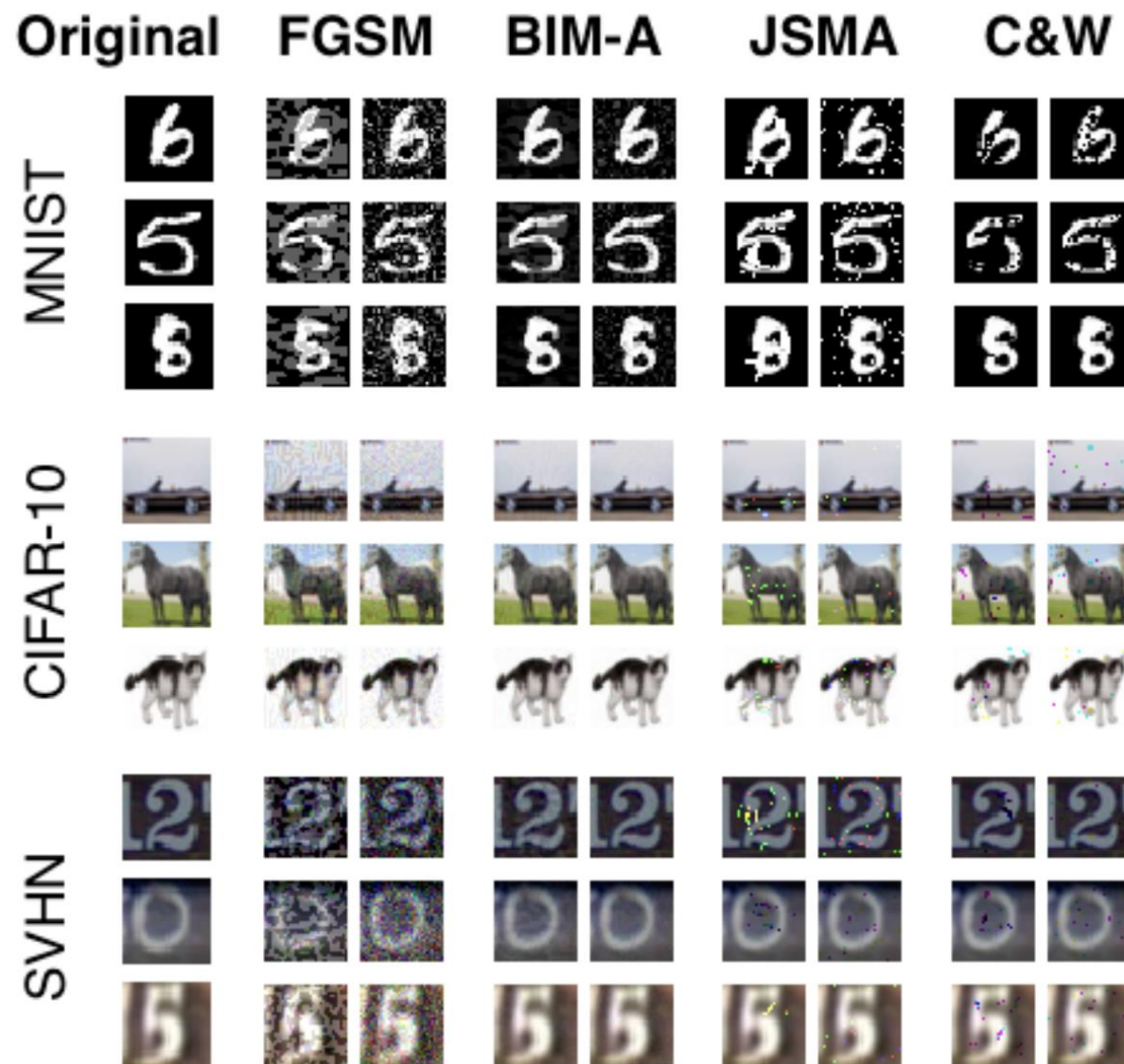
$\epsilon=0.05$

$\epsilon=0.1$

$\epsilon=0.2$

$\epsilon=0.3$

PGD CIFAR-10

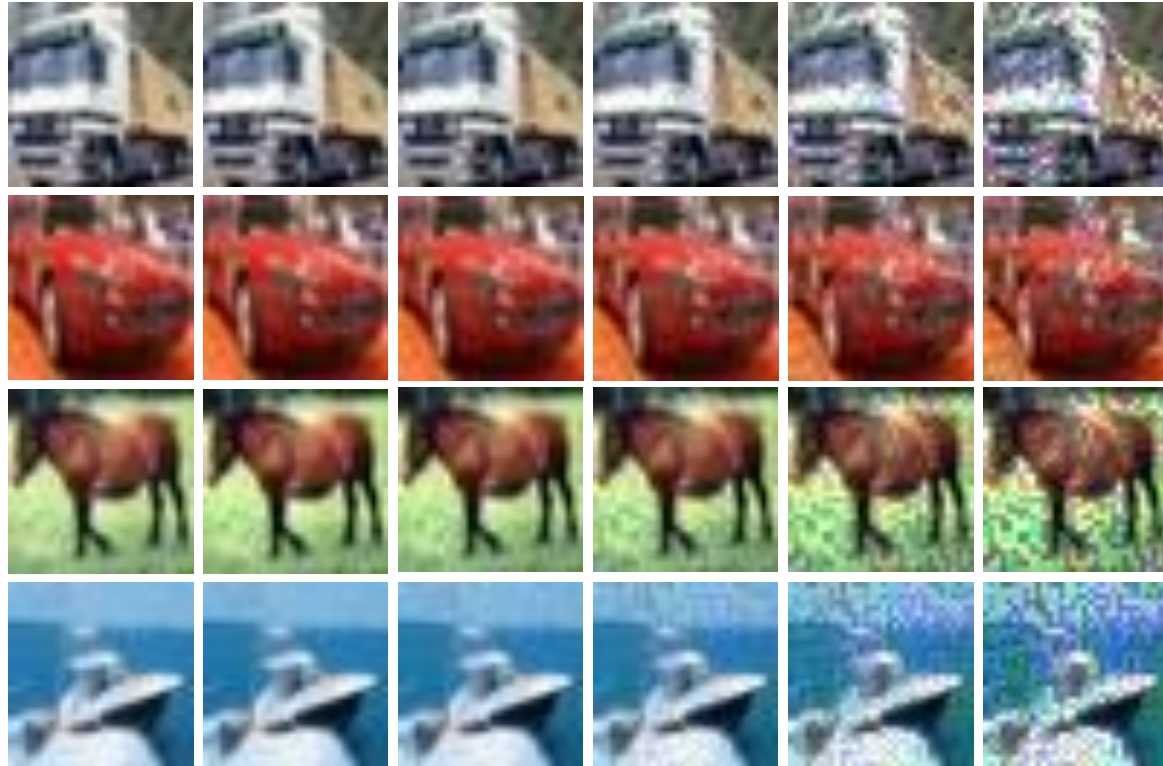


White-Box Attack

❖ Input-Space Adversarial Attack

- The perturbation is added to the raw data

$$\max_{\|\delta\|_p \leq \epsilon} \mathcal{L}(F_\theta(x + \delta), y_{true})$$



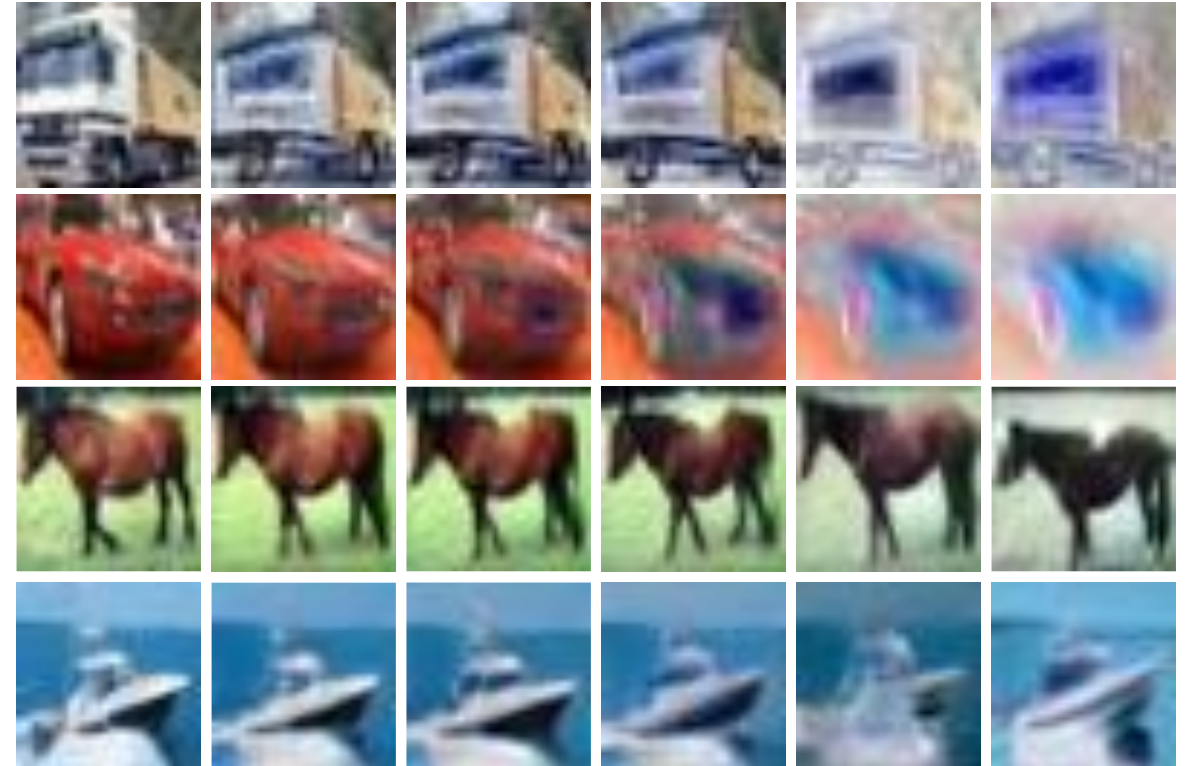
Clean $\epsilon=0.02$ $\epsilon=0.05$ $\epsilon=0.1$ $\epsilon=0.2$ $\epsilon=0.3$

PGD CIFAR-10

❖ Latent-Space Adversarial Attack

- Perturb the hidden representations

$$\max_{\|\zeta\|_p \leq \eta} \mathcal{L}(F_\theta(G_\phi(z + \zeta)), y_{true})$$

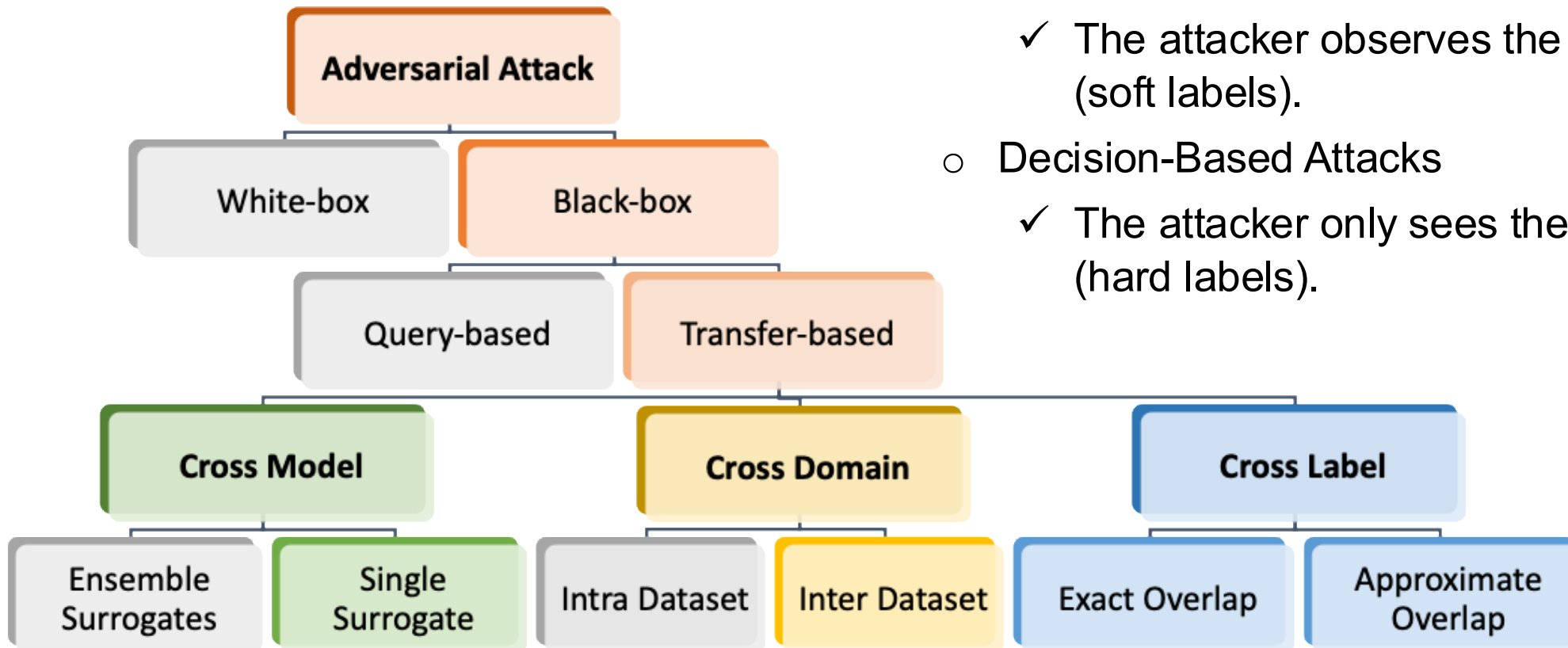


Clean $\eta=0.02$ $\eta=0.05$ $\eta=0.1$ $\eta=0.2$ $\eta=0.3$

OM-PGD CIFAR-10

❖ Black-box Attack

- Transfer-Based Attacks
 - The surrogate (substitute/proxy) model strategy
- Query-Based Attacks
 - Score-Based Attacks
 - ✓ The attacker observes the output probabilities (soft labels).
 - Decision-Based Attacks
 - ✓ The attacker only sees the final predicted label (hard labels).



Black-Box Attack

❖ Transfer-Based Attacks

- Adversarial examples generated for surrogate model A often fool victim model B.
 - Cross-Model Architecture Transfer
 - Most transfer attack studies enhance transferability by mitigating surrogate overfitting via gradient regularization (MI-FGSM) or input diversification (DI-FGSM).
 - ✓ MI-FGSM (Momentum): integrates a momentum term (i.e., accumulated gradient) into the basic iterative attack to stabilize the gradient update direction.

$$g^{t+1} = \mu \cdot g^t + \frac{\nabla_{x^t} \mathcal{L}(f(x^t), y)}{\|\nabla_{x^t} \mathcal{L}(f(x^t), y)\|_1} \quad x^{t+1} = \text{Clip}_{x, \epsilon} \{x^t + \alpha \cdot \text{sign}(g^{t+1})\}$$

- ✓ DI-FGSM (Diverse-input): A transformation function T (e.g, resizing, padding) applied with probability p . $x^{t+1} = \text{Clip}_{x, \epsilon} \{x^t + \alpha \cdot \text{sign}(\nabla_{x^t} \mathcal{L}(f(T(x^t; p)), y))\}$
 - Cross-Dataset Transfer
 - Using a pretrained model as the surrogate to attack its fine-tuned counterpart

- Dong, Yinpeng, Fangzhou Liao, Tianyu Pang, Hang Su, Jun Zhu, Xiaolin Hu, and Jianguo Li. "Boosting adversarial attacks with momentum." In Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 9185-9193. 2018.
- Xie, Cihang, Zhishuai Zhang, Yuyin Zhou, Song Bai, Jianyu Wang, Zhou Ren, and Alan L. Yuille. "Improving transferability of adversarial examples with input diversity." In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp. 2730-2739. 2019.

Black-Box Attack

❖ Query-Based Attacks

- Score-Based Attacks: The attacker observes the output probabilities (soft labels).
 - ZOO (Zeroth Order Optimization).
 - Instead of backpropagation, ZOO uses Finite Difference to estimate the gradient for each pixel (feature) x_i from the model's output probabilities. h is a small constant.

$$\hat{g}_i = \frac{\partial \mathcal{L}(x, y)}{\partial x_i} \approx \frac{\mathcal{L}(x + he_i, y) - \mathcal{L}(x - he_i, y)}{2h}$$

- Once the gradients are estimated, the attack employs C&W loss to find the perturbation.
 - The query complexity is high, making it difficult to scale to high-dimensional inputs.
 - Decision-Based Attacks: The attacker only sees the final predicted label (hard labels).
 - Boundary Attack
 - Start with an image already in the "target" class (e.g., a random noise image).
 - move the adversarial image closer to the original "clean" image.

- ❑ Chen, Pin-Yu, Huan Zhang, Yash Sharma, Jinfeng Yi, and Cho-Jui Hsieh. "Zoo: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models." In Proceedings of the 10th ACM workshop on artificial intelligence and security, pp. 15-26. 2017.
- ❑ Brendel, Wieland, Jonas Rauber, and Matthias Bethge. "Decision-based adversarial attacks: Reliable attacks against black-box machine learning models." arXiv preprint arXiv:1712.04248 (2017).

Part III | Empirical Defenses



Adversarial Training

❖ Empirical Defenses

- Heuristic-based method.
- Evaluated on the known attacks (like PGD, CW, or AutoAttack).

❖ Adversarial Training (The Gold Standard)

- It turns the training process into a **Min-Max** Optimization problem.
- Instead of training only on clean data, the model is trained on adversarial examples generated during each training step (batch-level).
- Formulation:

$$\min_{\theta} \mathbb{E}_{(x,y) \sim \mathcal{D}} \left[\max_{\delta \in \Delta} \mathcal{L}(f_{\theta}(x + \delta), y) \right]$$

- The Attack (Inner Maximization)
 - Find the “worst-case” perturbation δ within a budget Δ (via PGD) that maximizes the model's loss $\mathcal{L}$.
- The Defense (Outer Minimization)
 - Update the model parameters θ to minimize $\mathcal{L}$ on these "worst-case" examples.
- Limitation: usually leads to a drop in standard (clean) accuracy

Adversarial Training

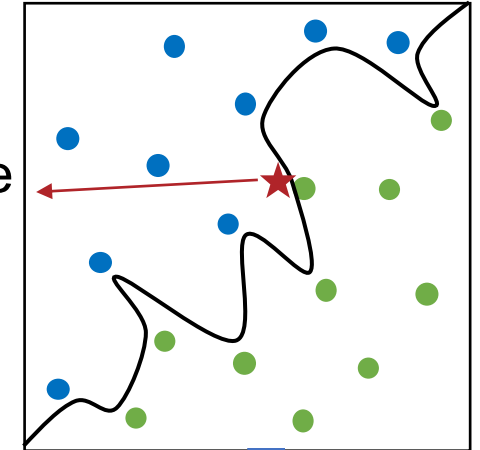
❖ Adversarial Training (The Gold Standard)

- The model is trained on adversarial examples generated during each training step (batch-level).

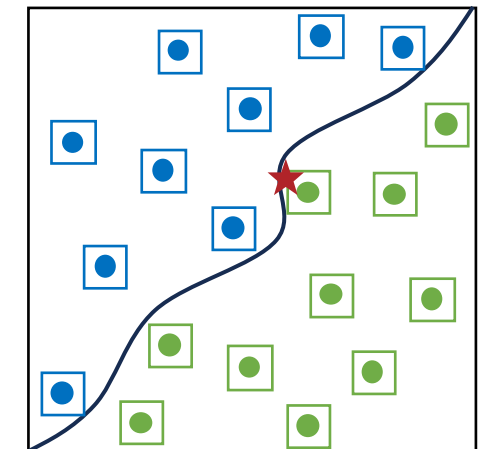
```
def train_adversarial(model, train_loader, optimizer, device, eps=8/255):  
    model.train()  
    for images, labels in train_loader:  
        images, labels = images.to(device), labels.to(device)  
  
        # 1. Generate adversarial examples (The Inner Max)  
        # Note: We usually keep the model in eval mode or disable batchnorm  
        # updates during attack generation to avoid polluting BN statistics.  
        model.eval()  
        adv_images = pgd_attack(model, images, labels, eps=eps)  
        model.train()  
  
        # 2. Update model parameters (The Outer Min)  
        optimizer.zero_grad()  
        outputs = model(adv_images)  
        loss = nn.CrossEntropyLoss()(outputs, labels)  
        loss.backward()  
        optimizer.step()
```

Adversarial Example

Standard Training



Adversarial Training



Variations of Adversarial Training

❖ TRADES (Theoretically Principled Robustness Accuracy Trade-off Selected)

- TRADES loss = a Natural Loss + a Robust Regularization term.

$$\min_{\theta} \mathbb{E} \left\{ \underbrace{\mathcal{L}(f_{\theta}(x), y)}_{\text{Natural Loss}} + \beta \cdot \underbrace{\max_{\delta \in \Delta} \mathcal{L}_{KL}(f_{\theta}(x), f_{\theta}(x + \delta))}_{\text{Invariance Regularization}} \right\}$$

- Natural Loss
 - A standard Cross-Entropy loss that ensures the model remains accurate on clean data.
- Invariance Regularization
 - Uses KL Divergence to force the model's prediction for an adversarial example $x + \delta$ to be as close as possible to its prediction for the clean example x .
- Trade-off Parameter β
- Compare with PGD-AT
 - In PGD-AT, the goal is to correctly classify the "worst" point. So the target is the truth label y .
 - In TRADES, the goal is to make predictions "stable" within the ϵ -ball. So the target is the model's prediction on x .

Variations of Adversarial Training

❖ MART (Misclassification Aware adveRsarial Training)

- MART treats *misclassified clean examples* and *correctly classified clean examples* differently
 - Standard AT and TRADES treat all data points equally.
 - Over-smoothing on easy samples leads to a drop in clean accuracy.
- MART Loss
$$\mathcal{L}_{MART} = \mathcal{L}_{BCE}(f_{\theta}(x + \delta), y) + \lambda \cdot \mathcal{L}_{KL}(f_{\theta}(x), f_{\theta}(x + \delta)) \cdot (1 - p_y(x))$$
- Boosted Cross-Entropy (BCE)
 - It not only targets the correct label y but boosts the decision margin by penalizing the most competitive incorrect class.
$$\mathcal{L}_{BCE} = -\log(p_y(x + \delta)) - \log(1 - \max_{k \neq y} p_k(x + \delta))$$
- Misclassification-Aware KL:
 - Adaptive weight $1 - p_y(x)$
 - If f is confident in the truth class y for the clean input x , the weight $1 - p_y(x)$ is low.
 - If f is wrong or uncertain, the weight $1 - p_y(x)$ is high.

Variations of Adversarial Training

❖ Accelerating AT: Free-AT (Free Adversarial Training)

- PGD-AT perform K steps of PGD to find the perturbation δ ,

$$x^{t+1} = \prod_{x+\Delta} (x^t + \alpha \cdot \text{sgn}(\nabla_{x^t} L(f_\theta(x^t), y)))$$

- PGD Step (iteration) number K : $t = 1, \dots, K$
- For each batch, $K + 1$ times gradient calculations are performed before updating θ .
- Free-AT addresses the computational overhead weakness of PGD-AT via Gradient Re-use
 - For T hops of Free-AT, calculate the gradient of the loss with respect to both θ and x .

$$\delta_{t+1} = \text{Clip}_\epsilon(\delta_t + \eta \cdot \text{sign}(\nabla_x \mathcal{L}(f_\theta(x + \delta_t), y)))$$

$$\theta_{t+1} = \theta_t - \gamma \cdot \nabla_\theta \mathcal{L}(f_\theta(x + \delta_t), y)$$

- PGD Step (iteration) number T : $t = 1, \dots, T$
- Model parameters are updated after each single-step perturbation update.
- The total number of training epochs can be reduced.

Variations of Adversarial Training

❖ Accelerating AT: Free-AT (Free Adversarial Training)

```
def train_free_at(model, train_loader, optimizer, device, eps=8/255, m=8):
    """
    m: The number of 'hops' or replays for each minibatch.
    eps: The adversarial budget.
    """
    model.train()

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        # Initialize delta to zero or random for the first hop
        # Note: delta is NOT reset during the m-hops
        delta = torch.zeros_like(images).uniform_(-eps, eps).requires_grad_(True)

        for _ in range(m):
            # 1. Forward pass with the current perturbation
            outputs = model(images + delta)
            loss = nn.CrossEntropyLoss()(outputs, labels)

            # 2. Backward pass: Calculates gradients for BOTH theta and delta
            optimizer.zero_grad()
            loss.backward()

            # 3. Simultaneous Update
            # Update weights (Outer Min)
            optimizer.step()

            # Update perturbation (Inner Max) using the gradient we already calculated
            with torch.no_grad():
                grad_delta = delta.grad.sign()
                delta.data = delta + eps * grad_delta # Using eps as step size is
                delta.data = torch.clamp(delta, -eps, eps)
                delta.data = torch.clamp(images + delta, 0, 1) - images

            # Reset delta grad for the next hop, but keep delta.data!
            delta.grad.zero_()
```

Feature	Madry PGD-AT	Free-AT
Updates per Batch	1 weight update after K attack steps.	K times weight updates and K attack steps.
Complexity	$\mathcal{O}(N \times K)$	$\mathcal{O}(N)$ (N is total updates number).
Efficiency	Slow	Fast
Robustness	High	Slightly lower but comparable

Gradient Regularization / Preprocessing / Detection

❖ Gradient Regularization

- Adds a penalty to the cross-entropy loss that forces the Jacobian matrix $\nabla_x(f(x))$ to be small.

$$\mathcal{L}_{CE}(f(x), y) + \lambda \cdot \|\nabla_x \mathcal{L}(f(x), y)\|_2^2$$

❖ Input Preprocessing

- Remove (purify) adversarial noise before the data reaches the classifier.
 - Denoising Autoencoders
 - Denoiser (e.g., U-Net, PixelCNN) is trained to map adversarial input back to clean data.
 - Feature Squeezing
 - Reduces the search space available to an attacker by reducing color bit depth.

❖ Detection-based Defense

- Identify and reject adversarial example.
 - A detector is trained specifically to distinguish between clean and adversarial inputs.

- Ross, Andrew, and Finale Doshi-Velez. "Improving the adversarial robustness and interpretability of deep neural networks by regularizing their input gradients." In Proceedings of the AAAI conference on artificial intelligence, vol. 32, no. 1. 2018.
- Song, Yang, Taesup Kim, Sebastian Nowozin, Stefano Ermon, and Nate Kushman. "Pixeldefend: Leveraging generative models to understand and defend against adversarial examples." arXiv preprint arXiv:1710.10766 (2017).
- Liao, Fangzhou, Ming Liang, Yinpeng Dong, Tianyu Pang, Xiaolin Hu, and Jun Zhu. "Defense against adversarial attacks using high-level representation guided denoiser." In Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 1778-1787. 2018.

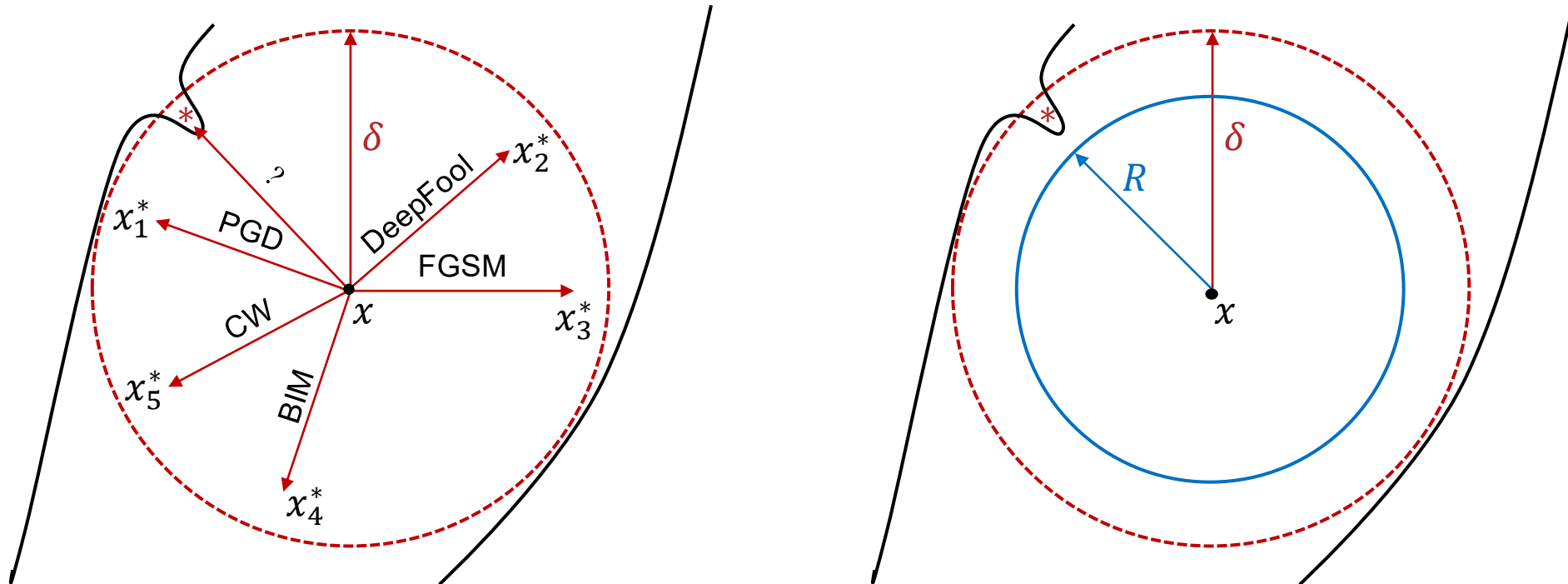
Part IV | Certified Robustness



Motivation of Certified Robustness

❖ Breaking the Arms Race

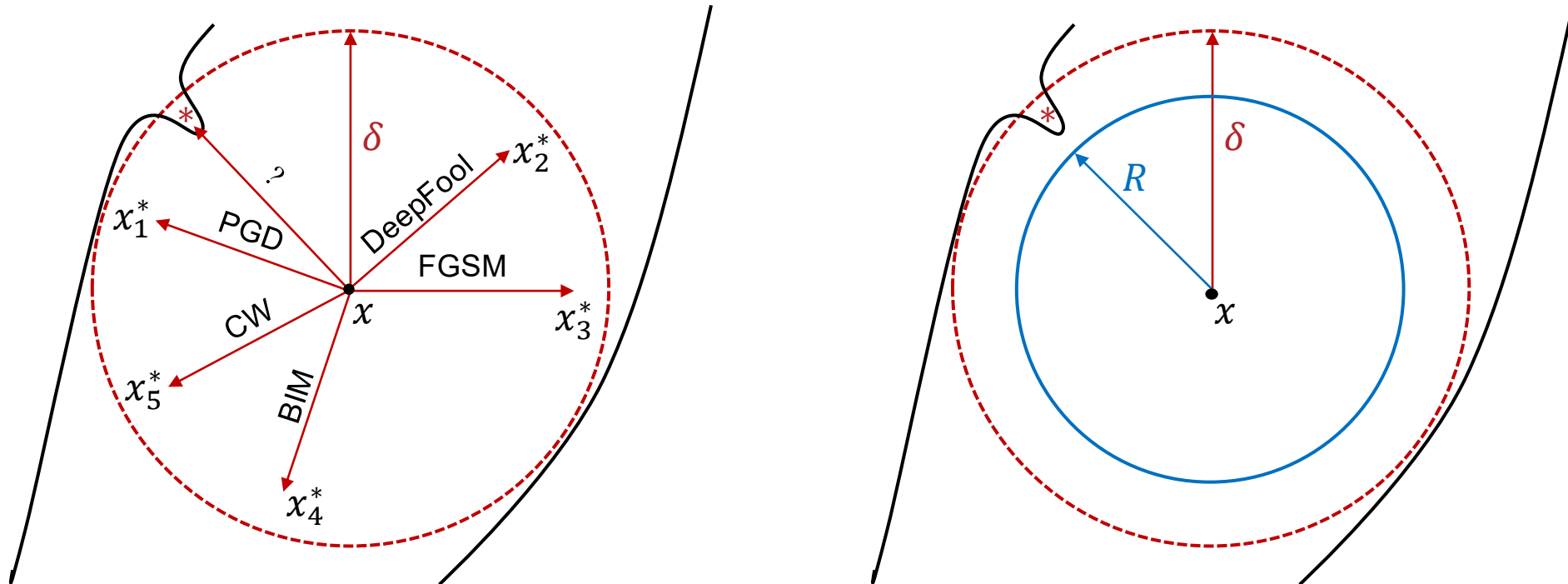
- In adversarial ML, every empirical defense has eventually been bypassed by a more sophisticated attacker, or a new optimization strategy is used.
- Empirical Defenses
 - They work against "known" attacks used during training but often fail to adaptive attacks (attacks specifically designed to bypass that defense).



Motivation of Certified Robustness

❖ Breaking the Arms Race

- In adversarial ML, every empirical defense has eventually been bypassed by a more sophisticated attacker, or a new optimization strategy is used.
- **Certified Robustness**
 - Provide a guarantee that is independent of the attack algorithm.
 - *If an input x is certified robust for a radius ϵ* , no attack can flip the prediction label within that input range, regardless of how it is optimized.



Motivation of Certified Robustness

❖ Breaking the Arms Race

- In adversarial ML, every empirical defense has eventually been bypassed by a more sophisticated attacker, or a new optimization strategy is used.
- **Certified Robustness**
 - Provide a guarantee that is independent of the attack algorithm.
 - *If an input x is certified robust for a radius ϵ* , no attack can flip the prediction label within that input range, regardless of how it is optimized.
 - Two major benefits
 - Quantifiable Trust
 - Calculate a verifiable distance (the Certified Radius) within which the model's decision is $\sim 100\%$ stable.
 - Generalization to Unseen Perturbations
 - Certified defenses provide a framework that can generalize better to unknown attacks, natural corruptions, and random noise, ensuring the model isn't just overfitting to a specific type of attack.

Motivation of Certified Robustness

❖ Breaking the Arms Race

- In adversarial ML, every empirical defense has eventually been bypassed by a more sophisticated attacker, or a new optimization strategy is used.
- **Certified Robustness**
 - Provide a guarantee that is independent of the attack algorithm.
 - *If an input x is certified robust for a radius ϵ* , no attack can flip the prediction label within that input range, regardless of how it is optimized.

Feature	Empirical Defense (e.g., Adversarial Training)	Certified Defense (e.g., Randomized Smoothing)
Security Guarantee	Heuristic (works until a better attack is found)	Provable (mathematically guaranteed)
Attack Independence	No (sensitive to attack parameters)	Yes (independent of the attack method)
Computational Cost	Moderate (training is slow)	High (verification is expensive)

Formalization of Certified Robustness

❖ Base Model

DEFINITION 1 (BASE CLASSIFIER). Given a DNN model f_θ that maps input samples x to a C -dimensional confidence score vector, a base classifier F is defined as:

$$F(x) = \arg \max_{i \in [C]} f_\theta(x)_i,$$

where $f_\theta(x)_i$ represents the confidence score of the model f_θ on the i -th category for a given input x .

❖ Robustness Guarantee

- The robustness guarantee provided by a classifier for an input x is formalized as the largest L_p -bounded certified radius R such that the verification algorithm can prove no adversarial example exists within that distance.

DEFINITION 2 (l_p ROBUSTNESS GUARANTEE). Given a classifier F , an input x , a perturbation δ , a distance measure l_p , robustness guarantee of F on x against l_p -bounded δ is defined as:

$$\text{For all } \delta \text{ such that } \|\delta\|_p < R, F(x + \delta) = F(x),$$

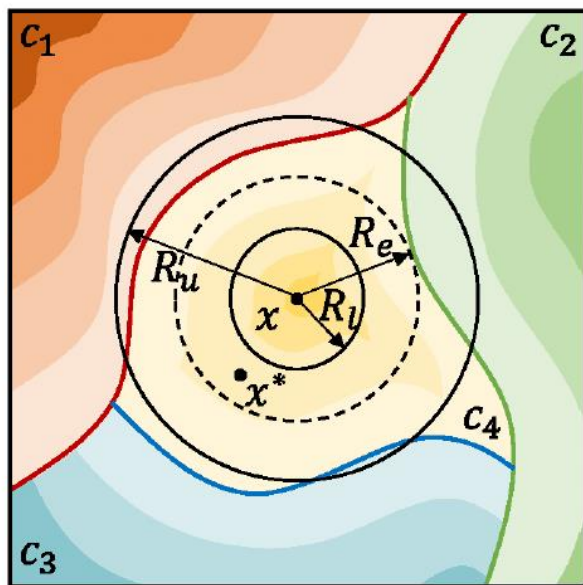
where R is the certified radius.

Types of Norm Robustness Guarantees

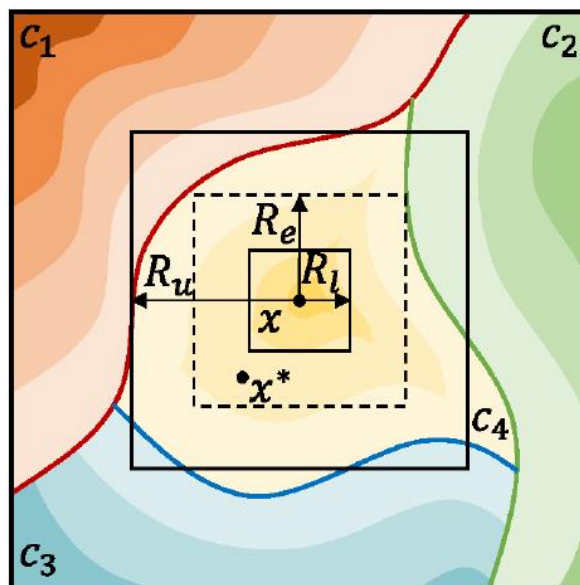
❖ L_p Norm

- Norm used in certification defines how the attacker is allowed to manipulate the input clean data.

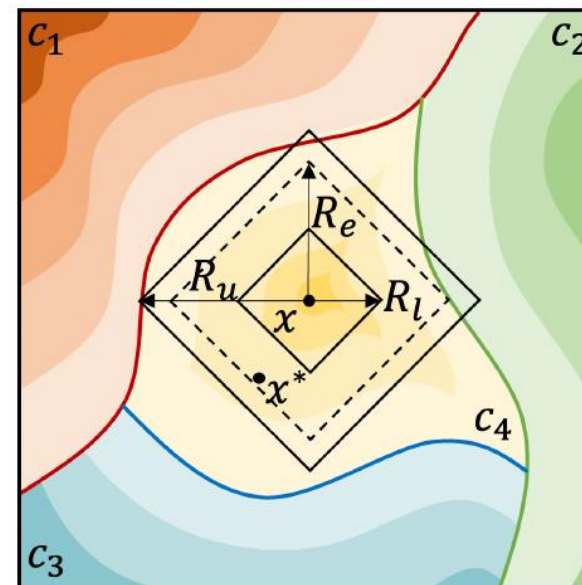
Norm Type	Geometry of Certified Input Area	Primary Certification Tool
L_2	Circle / Sphere / Hypersphere	Randomized Smoothing (Gaussian)
L_∞	Square / Cube / Hypercube	Interval Bound Propagation, Linear Relaxation
L_1	Diamond / Octahedron / Cross-polytope	Randomized Smoothing (Laplace)



(a) l_2 radius: $\|\delta\|_2 < R$



(b) l_∞ radius: $\|\delta\|_\infty < R$

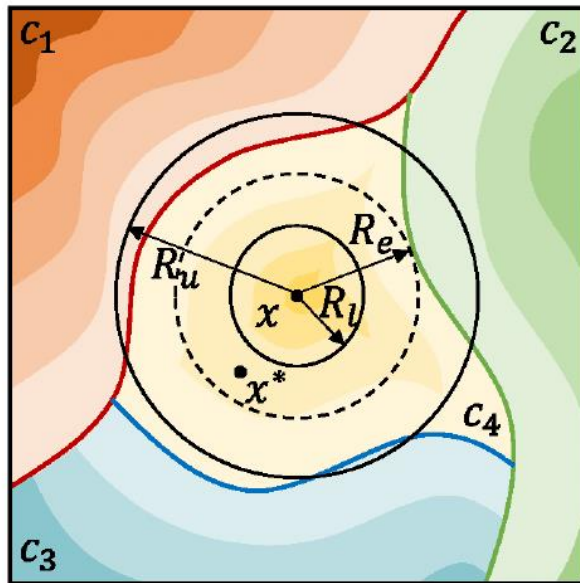


(c) l_1 radius: $\|\delta\|_1 < R$

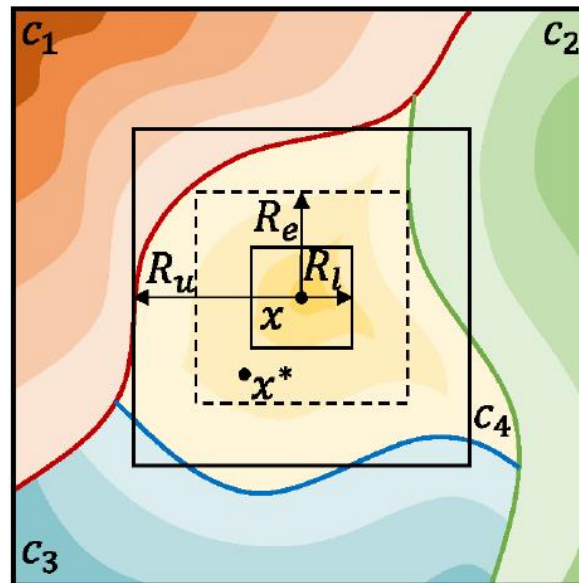
Certified Radius

❖ Robust Radius

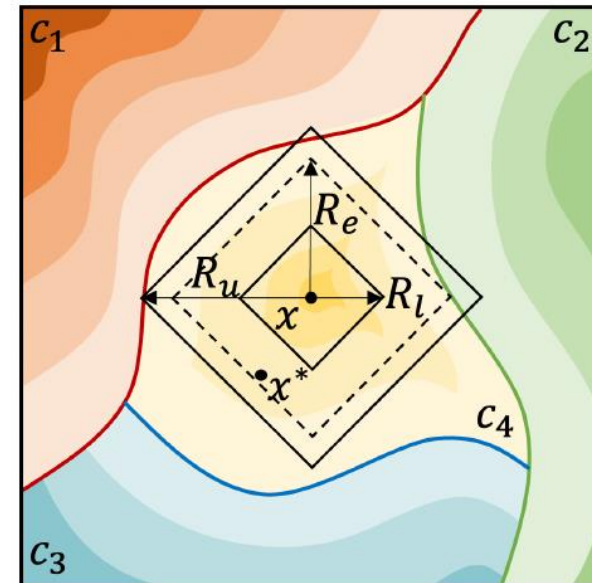
- The Ground Truth
 - Exact Robust Radius (Dashed line) R_e
 - True limit: maximum distance from an input x to the nearest decision boundary.
 - Finding R_e is an NP-complete problem for general NNs. Too slow for large-scale models.
 - Finding this value is the goal of **Complete Certification**.



(a) l_2 radius: $\|\delta\|_2 < R$



(b) l_∞ radius: $\|\delta\|_\infty < R$

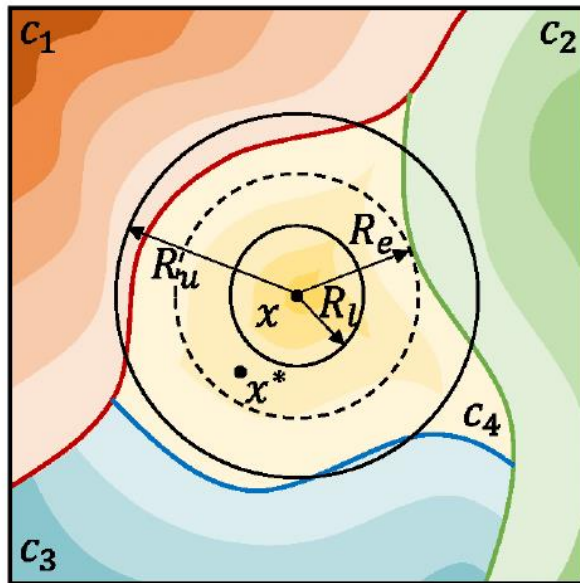


(c) l_1 radius: $\|\delta\|_1 < R$

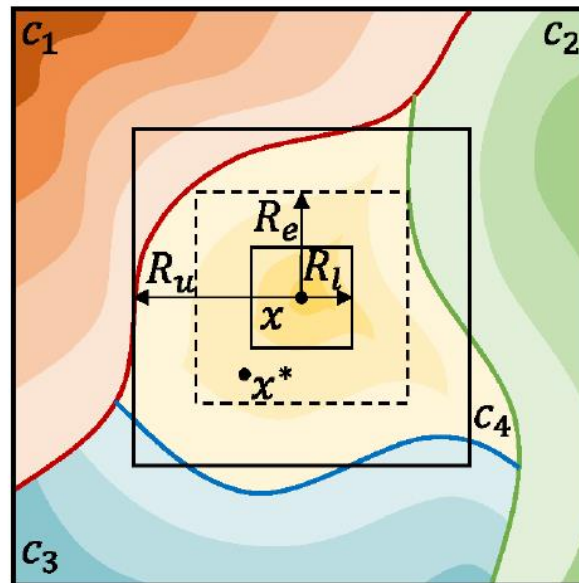
Certified Radius

❖ Robust Radius

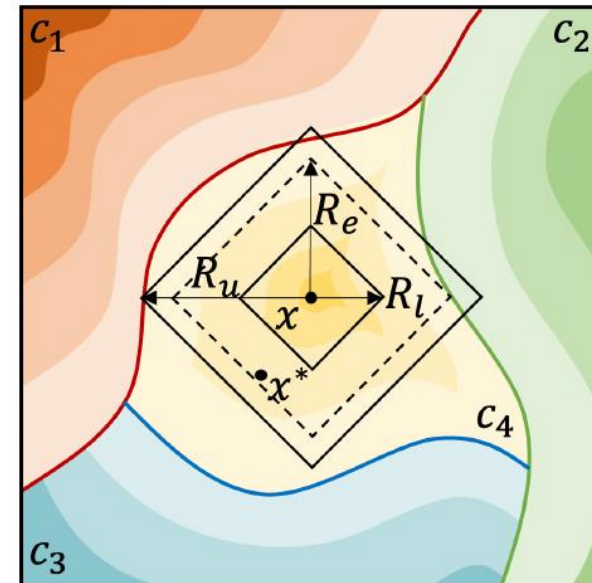
- Lower Bound of Exact Robust Radius (Solid Line) R_l
 - R_l is the most critical metric in certified defense because it provides the guarantee.
 - It defines a "safe zone." We are 100% certain (or highly certain in probabilistic models) that no adversarial attack can succeed within this radius.
 - Finding this value is the goal of [Incomplete Certification](#) methods like Interval Bound Propagation (IBP) or Randomized Smoothing.



(a) l_2 radius: $\|\delta\|_2 < R$



(b) l_∞ radius: $\|\delta\|_\infty < R$

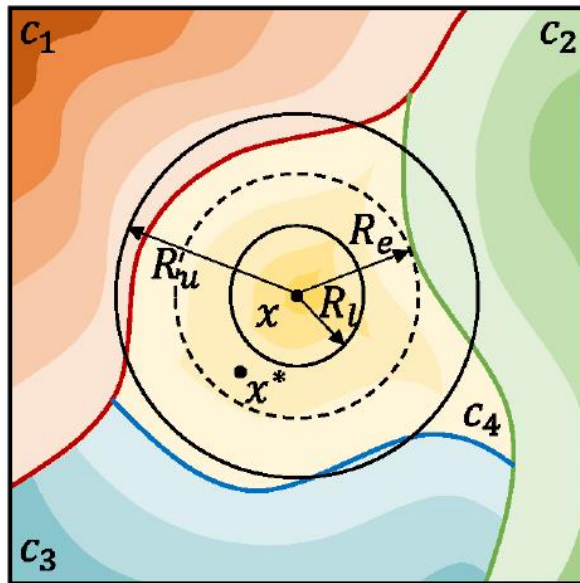


(c) l_1 radius: $\|\delta\|_1 < R$

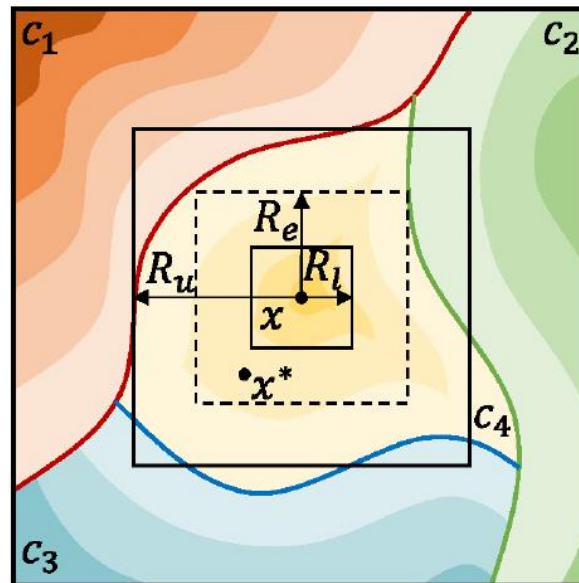
Certified Radius

❖ Robust Radius

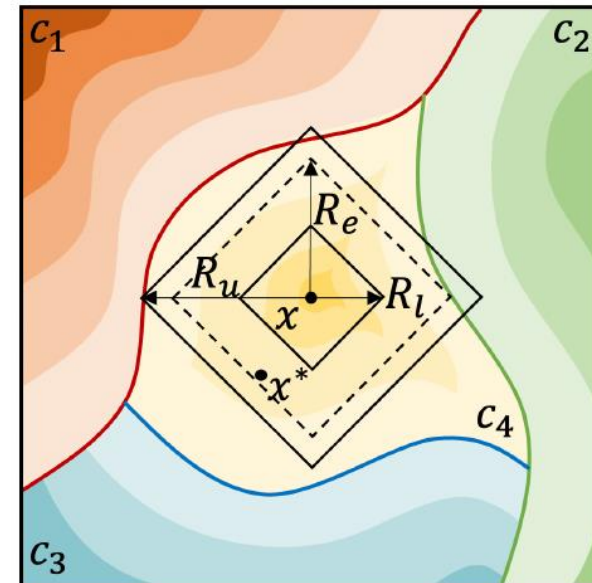
- Upper Bound of Exact Robust Radius (Solid Line) R_u
 - R_u represents a distance at which we know the model is not robust.
 - If an adversarial attack (like PGD) successfully finds a perturbation at distance ϵ , then ϵ is an upper bound on the true radius R_e .
 - The upper bound is found via adversarial attacks.



(a) l_2 radius: $\|\delta\|_2 < R$



(b) l_∞ radius: $\|\delta\|_\infty < R$

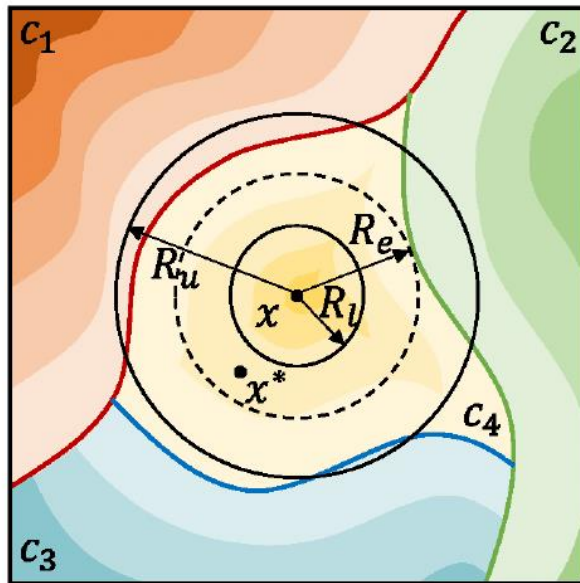


(c) l_1 radius: $\|\delta\|_1 < R$

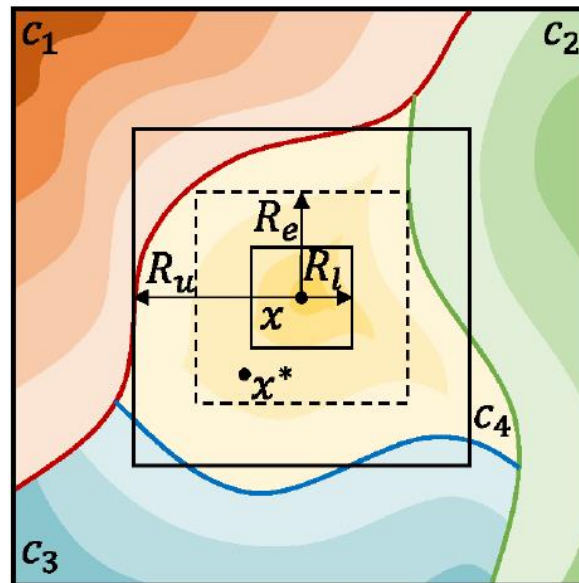
Certified Radius

❖ Robust Radius

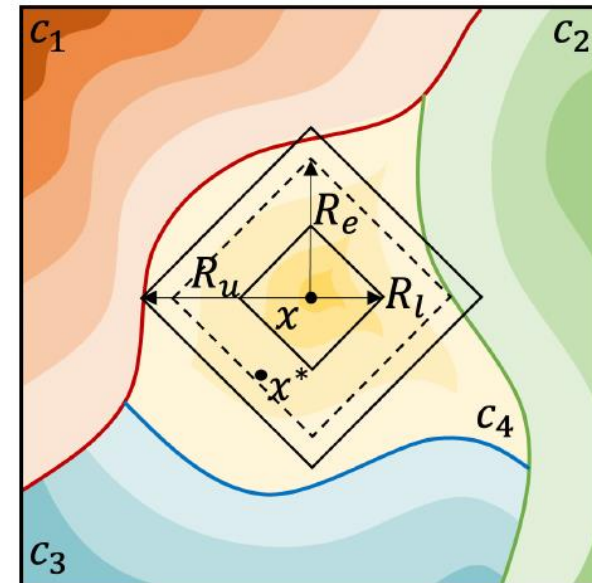
Metric	Symbol	Meaning	How it's Found
Lower Bound	R_l	"It is at least this robust."	Certified Defenses (IBP, Smoothing)
Exact Radius	R_e	"The true distance to failure."	Exact Solvers (SMT, MILP)
Upper Bound	R_u	"It is at most this robust."	Adversarial Attacks (PGD, CW)



(a) l_2 radius: $\|\delta\|_2 < R$



(b) l_∞ radius: $\|\delta\|_\infty < R$



(c) l_1 radius: $\|\delta\|_1 < R$

Taxonomy of Certification Method

❖ Complete vs. Incomplete Certification

- Categorized based on how the method searches for vulnerabilities
- **Complete** Certification: Provide exact radius R_e
 - Explore the entire input space in the L_p ball to determine if an adversarial example exists.
 - If the algorithm fails to find an adversarial example, it is certain that none exists.
 - Tools: SMT (Satisfiability Modulo Theories) solvers or MILP (Mixed Integer Linear Program).

Input	Output Label	Certification Result	Complete Verification	Incomplete Verification
x	y	"verified" (pass)	Definitely safe	Definitely safe
x	y	"not verified" (fail)	Definitely unsafe (AE certainly exists)	May be safe or unsafe (method is inconclusive about x)

- **Incomplete** Certification: Provide a lower bound radius R_l
 - If x is certified robust, it is definitely safe;
 - If x cannot be certified, x might still be robust in reality. It was too conservative to prove it.
 - Tools: Interval Bound Propagation (IBP) and Randomized Smoothing.

Taxonomy of Certification Method

❖ Deterministic vs. Probabilistic Certification

- Categorized based on the mathematical certainty.
- **Deterministic** Certification:
 - Under the specified constraints, it is 100% certain the model's output will not change.
 - Analyzes the target model directly
 - Tools: Linear Relaxations or Exact Solvers.

Input	Output	True Robustness	Deterministic Verification	Probabilistic Verification
x	y	Robust (Safe)	Always outputs "verified"	Outputs "verified" with high probability
x	y	Non-robust (Unsafe)	Always outputs "not verified"	Outputs "not verified" with high probability

- **Probabilistic** Certification:
 - The model is robust with a very high probability (e.g., 99%), based on statistical sampling.
 - Instead of certifying the base model $f(x)$, it certifies a smoothed model $f_{smo}(x)$ by adding noise (e.g., Gaussian) to the input and taking a majority vote.
 - Tools: Randomized Smoothing.

Formalization of Certified Robustness

❖ Base Model

DEFINITION 1 (BASE CLASSIFIER). Given a DNN model f_θ that maps input samples x to a C -dimensional confidence score vector, a base classifier F is defined as:

$$F(x) = \arg \max_{i \in [C]} f_\theta(x)_i,$$

where $f_\theta(x)_i$ represents the confidence score of the model f_θ on the i -th category for a given input x .

❖ Smoothed Model

DEFINITION 3 (SMOOTHED CLASSIFIER). Given a base classifier F , an input x , a smoothing distribution $\mathcal{D}$ with Probability Density Function (PDF) φ , smoothed classifier F_{smooth} is defined as:

$$F_{smooth}(x) = \arg \max_{c \in [C]} \mathbb{P}_{\eta \sim \mathcal{D}}(F(x + \eta) = c) = \arg \max_{c \in [C]} \int_{\eta \sim \mathcal{D}} \mathbb{I}[F(x + \eta) = c] \varphi(\eta) d\eta,$$

where $\mathbb{P}$ is the probability function, $\mathbb{I}$ is the indicator function that returns 1 when the condition is true, and $\varphi(\eta)$ denotes the probability density of the sampled noise data η .

Taxonomy of Certification Method

❖ Comparison

Category	Strengths	Limitations
Complete	▪ Perfect Precision. ▪ No False Negatives	▪ Computationally expensive (NP-complete). ▪ Restricted to tiny networks (e.g., a few layers).
Incomplete	▪ High Speed. ▪ Works on medium-to-large architectures.	▪ Loose: Provides a lower bound ▪ False Failures: May fail to certify a robust point.
Deterministic	▪ 100% Certainty. ▪ Certifies the actual model you trained.	▪ Architecture Sensitive: Requires specific activation layers (like ReLU) to work effectively. ▪ Often difficult to implement for complex DNNs.
Probabilistic	▪ The only way to certify large-scale models. ▪ Doesn't need to know the model's internal weights.	▪ Guarantee holds with high probability not 100%. ▪ Requires hundreds or thousands of forward passes.

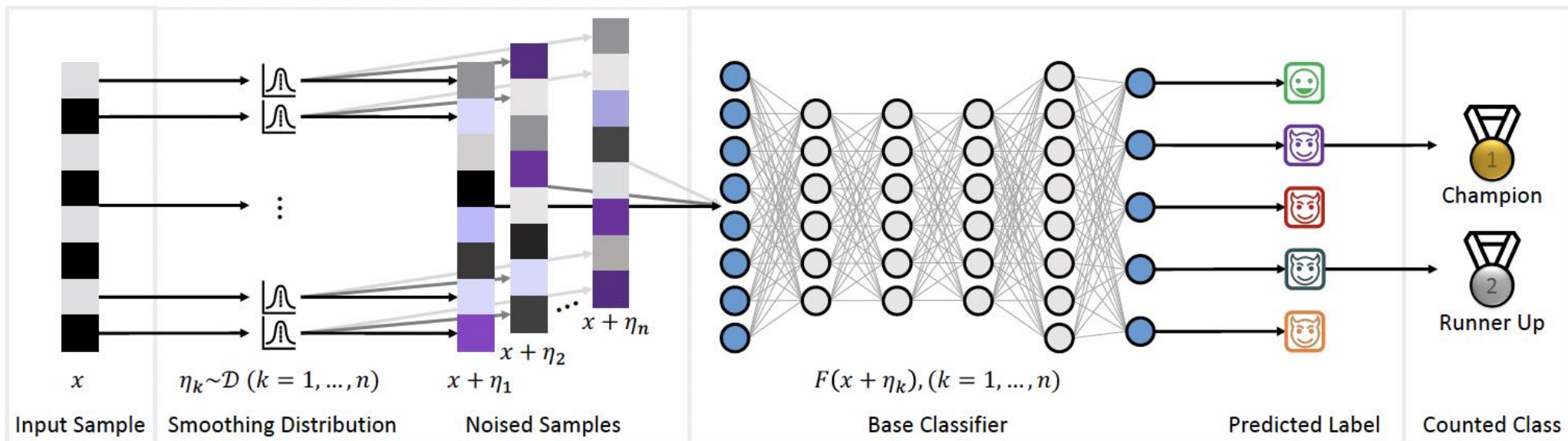
Randomized Smoothing

❖ Type

- An incomplete-probabilistic certification that provides guarantees through a **smoothed classifier**.

❖ Mechanism

- It takes a x , adds random Gaussian noise, and takes a majority **vote** of the predictions.
- It calculates a R_l within which the prediction is guaranteed to be stable with high probability.
- Prediction: Sampling $n_k = n_{small}$ noise data $\rightarrow$ Predict the category of the input x .
- Certification: Sampling $n_k = n_{large}$ noise data $\rightarrow$ Calculate the radius of the model on the input x .



Randomized Smoothing

❖ Type

- An incomplete-probabilistic certification that provides guarantees through a **smoothed classifier**.

❖ Mechanism

- It takes a x , adds random Gaussian noise, and takes a majority **vote** of the predictions.
- It calculates a R_l within which the prediction is guaranteed to be stable with high probability.
- Prediction: Sampling $n_k = n_{small}$ noise data $\rightarrow$ Predict the category of the input x .
- Certification: Sampling $n_k = n_{large}$ noise data $\rightarrow$ Calculate the radius of the model on the input x .

❖ Pros

- The only method that currently scales to full-resolution ImageNet models.
- It is architecture-agnostic and does not require gradients.

❖ Cons

- Requires several forward passes for a single certification.
- It provides a probabilistic rather than a 100% deterministic guarantee.

Interval Bound Propagation (IBP)

❖ Type

- An incomplete-deterministic certification that provides a provable lower bound on a model's robustness by **propagating input intervals through the network**.

❖ Formulation

- The goal is to compute an **output interval** $[z_L, \bar{z}_L]$ that contains all possible results for an input within an L_∞ ball $B_\epsilon(x) = \{x^*, ||x^* - x||_\infty \leq \epsilon\}$.

❖ Step-by-Step Propagation

- Input Layer Start with the input region box
- Linear Layers
 - Multiply the positive weights by the input lower bounds and negative weights by the input upper bounds
 - Multiply the positive weights by the input upper bounds and negative weights by the input lower bounds:
- Activation Layers (e.g., ReLU)

$$[\underline{x}, \bar{x}] = [x - \epsilon, x + \epsilon]$$

$$z_i = W_i z_{i-1} + b_i$$

$$\underline{z}_i = W_i^+ \underline{z}_{i-1} + W_i^- \bar{z}_{i-1} + b_i$$

$$\bar{z}_i = W_i^+ \bar{z}_{i-1} + W_i^- \underline{z}_{i-1} + b_i$$

$$\underline{a}_i = \text{ReLU}(\underline{z}_i) \quad \bar{a}_i = \text{ReLU}(\bar{z}_i)$$

Interval Bound Propagation (IBP)

❖ Type

- An incomplete-deterministic certification that provides a provable lower bound on a model's robustness by [propagating input intervals through the network](#).

❖ Formulation

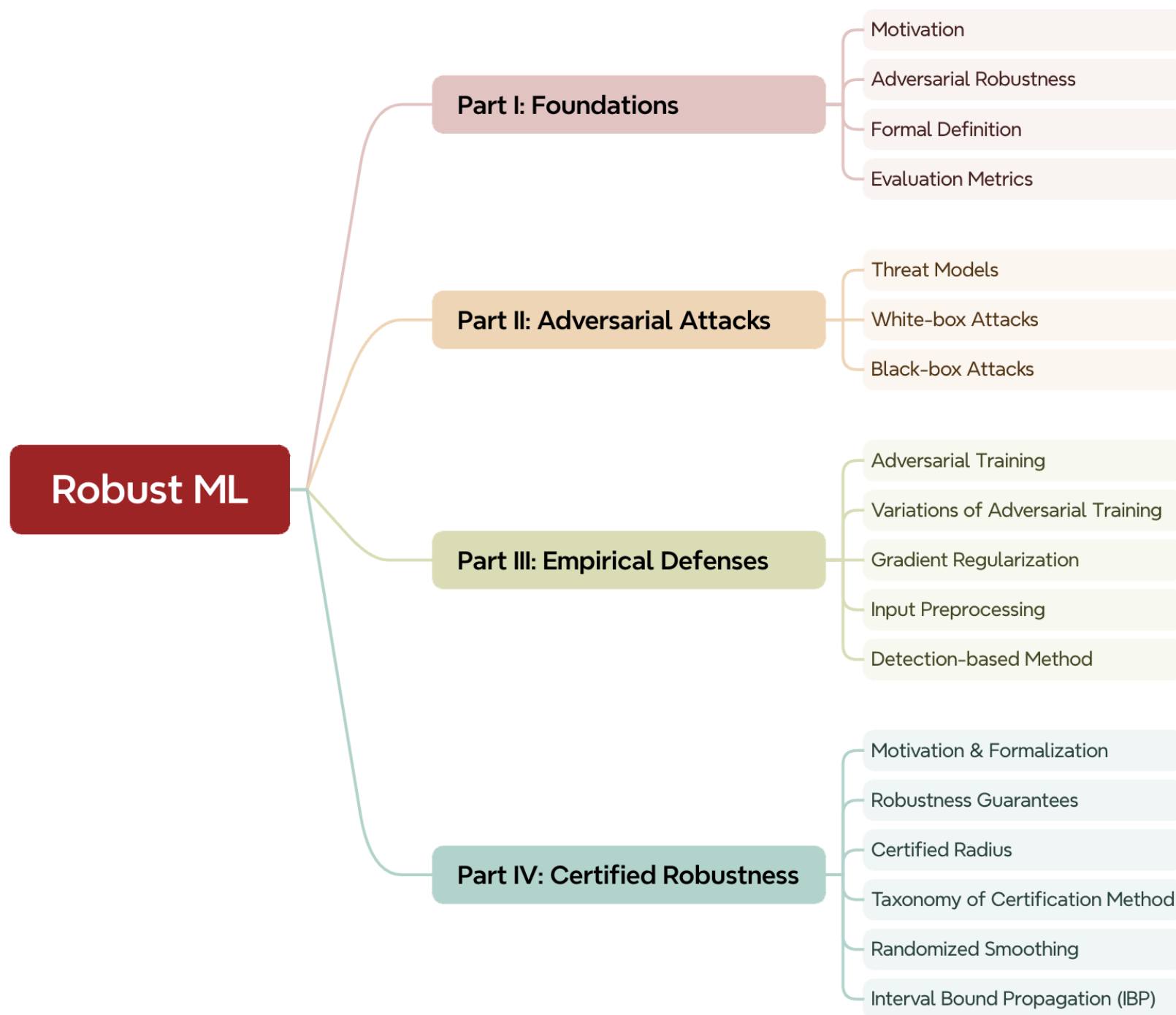
- The goal is to compute an [output interval](#) $[z_L, \bar{z}_L]$ that contains all possible results for an input within an L_∞ ball $B_\epsilon(x) = \{x^*, ||x^* - x||_\infty \leq \epsilon\}$.

❖ Pros

- The bounds are fully differentiable, allowing for Certified Training where the model minimizes the "certified loss" (worst-case error).
- It is the primary method for training verifiably robust models on datasets like CIFAR-10.
- IBP has the same complexity as a standard forward pass, making it fast.

❖ Cons

- IBP assumes each neuron is independent, which ignores correlations between neurons, leading to 0% certifying accuracy for deep models.



Q&A